Task1

Methodology

Task 1 required a 5-layer CNN for Fashion-MNIST classification, taking 1x28×28 greyscale images and producing a 10-class output. I implemented two models: a baseline CNN and a second version with Batch Normalisation (BN). The NN used five 3×3 convolutional layers with stride 1 and padding 1, increasing channel depth from 32 to 64, 128, 256 and 512. This follows lecture material showing that stacked small filters are effective because they increase the receptive field gradually while keeping the architecture simple and parameter-efficient. Max-pooling was applied after earlier blocks to reduce spatial dimensions and computation, whilst retaining salient features.

ReLU was used after each convolution. Sigmoid and tanh saturate and cause small derivatives that weaken backpropagated gradients. ReLU maintains stronger gradient propagation for positive activations and typically learns much faster in deep CNNs. The model depth was sufficient for optimisation stability to matter, especially when comparing the baseline model against the BN version.

Global Average Pooling (GAP) followed by 30% dropout and a final linear layer are used to reduce parameter count before classification. GAP is a replacement to large fully connected heads by averaging each final feature map. This produces a compact representation that is less parameter heavy and easier to generalise. Dropout with p=0.3 was included before the final classifier to reduce co-adaptation between neurons and help limit overfitting.
The input data were converted to tensors and normalised using the Fashion-MNIST mean and standard deviation. Standardisation improves optimisation by making activations more stable across layers. The original 60000 image training split was divided into 80% training and 20% validation examples using a fixed seed of 42 for reproducibility. The standard 10000 image test set was kept separate for final evaluation after training.

A learning-rate search was then carried out over five Adam LR: 10^−4, 5×10^−4, 10^−3, 5×10^−3, and 10^−2. Adam was chosen because it is a strong practical default that combines momentum with adaptive per-parameter updates, making it robust when curvature differs across parameters. The best learning rate was selected using validation

accuracy after five epochs. That value was used for both the baseline and BN experiments to ensure fairness in comparisons.
To evaluate the effect of BN, both models were trained for 15 epochs. The training and validation loss and accuracy were recorded after each epoch. The BN layers were inserted after each convolution and before ReLU. BN improves gradient flow, allows higher learning rates, reduces sensitivity to initialisation, and also acts as a mild regulariser.

Results

The learning-rate search in Figure 1 showed Adam 10^-3 produced the highest validation accuracy (92.5% at epoch 5) in the short search procedure. 10^-4 improved the most slowly of all five rates, reaching only 91.2% by epoch 5, matches with how an under-aggressive parameter updates the learning progress. 5×10^-4 performed similarly to 10^-3 in early epochs but fell behind from epoch 3 onwards, finishing at 92.3%. 5×10^-3 and 10^-3 at epoch 1 are comparable at epoch 1, but 5×10^-3 plateaued earlier and finished at 92.2%. 10^-2 was the only rate to show a regression between epoch 3-4, although recovering at epoch 5, has the overall lowest accuracy compared to other LRs. THis suggests the larger lr might cause the optimiser to overshoot the model.

Figure 2A shows the training and validation cross-entropy loss over 15 epochs for both the baseline and BN models. The BN model loss decreases more steeply in early epochs and reaches a lower final training loss than the baseline, indicating faster and more effective optimisation.

The baseline model finished with training accuracy of 98.0% and validation accuracy around 91.8%, whereas the BN model reached about 98.6% training accuracy and 92.2% validation accuracy. The BN model reached 92.5% validation accuracy by epoch 4, while the baseline reached 91.9% by epoch 5, indicating BN converges to a higher validation accuracy one epoch earlier.

In the baseline model, gradients pass through five convolutional layers without intermediate normalisation, so the scale of activations drifts during training and makes optimisation harder. In contrast, BN normalises batch activations and then learns a useful rescaling, which stabilises the distribution seen by later layers and makes parameter updates more reliable.

Figure 2B shows an increasing gap between training and validation accuracy in the BN model after epoch 10, reaching a training accuracy

of 98.6% versus 92.2% validation by epoch 15 . Although BN improved optimisation and final validation accuracy, the training accuracy continued to rise more strongly than validation accuracy, indicating some overfitting. Dropout appears to have limited this effect rather than eliminating it entirely, which is reasonable given the model capacity and the relatively simple dataset. A plausible improvement would be stopping training earlier or adding data augmentation.

Figure 3A shows the overall test accuracy of both models on the held-out test set. Both the baseline (91.63%) and the BN model (91.60%) achieved nearly identical test accuracy, despite BN (98.6%) has a slightly higher training accuracy than that of the baseline model(98.0%). This confirms the additional optimisation benefit of BN did not translate into a better generalisation on this data set.

Figure 3B indicates that some Fashion-MNIST classes were easier than others. Classes with clearer visual structure, such as footwear or bags, generally achieved stronger class-wise performance. Upper garments with similar silhouettes were weaker. These classes share overlapping silhouettes and texture patterns, making their feature representations harder to separate.

The confusion matrix in Figure 4 supports this interpretation. Misclassifications are concentrated among similar clothing categories, suggesting that the model struggled with inter class boundaries. High overall accuracy does not mean all categories are easy to distinguish.

Discussion

Overall, the experiment showed that Batch Normalisation improves optimisation but not necessarily generalisation. BN improved training stability and convergence speed, but did not produce a meaningful generalisation advantage at test time. Both models achieved 91.6% test accuracy. This suggests the baseline was already well-regularised enough for this dataset.

The architecture choices were intentionally conservative. Small 3×3 convolutions, ReLU nonlinearities, max pooling and Global Average Pooling provided a strong baseline without introducing unnecessary complexity.

However, the experiment also showed that optimisation does not equal generalisation. Both models showed a training-validation gap by epoch

15, with BN reaching 98.6%/92.2% and the baseline 98.0%/91.8%, indicating a degree of overfitting in both cases. This explains the faster convergence observed in the BN model early in training. Alternatively, the lower per-class accuracy for shirt-like garments (Figure 3B) reflects the difficulty caused by feature similarity between visually similar classes.

Task 2

Methodology

Task 2 required building and training an unconditional Generative Adversarial Network (GAN) to generate Fashion-MNIST images, then using both the discriminator and the Task 1 classifier to assess generated image quality.

A GAN consists of two networks trained adversarially, a generator (G in the code cell) that maps a random noise vector z ~ N(0,1) to a fake image, and a discriminator (D in the code cell) that outputs the probability that a given image is real. G learns to produce images that fool D, while D learns to distinguish real images from G output. At Nash equilibrium, D can no longer discriminate between real and generated images, and output 0.5 for both.

G takes a 32-dimensional latent noise vector as input. A fully connected projection layer maps this to 128×7×7, followed by BatchNorm1d and ReLU. Two transposed convolutional blocks upsample the spatial dimensions, each with kernel size 4, stride 2 and padding 1. The channel depth reduces from 128 to 64 in the first block, and from 64 to 1 in the second, progressively compressing the representation into a single-channel image. BatchNorm2d and ReLU follow the first upsample block. A tanh activation follows the second block, mapping output pixels to [−1, 1] to match the normalised real image range. D uses two strided convolutional layers. The first layer downsamples the input from 1 channel at 28×28 to 64 feature maps at 14×14, and the second reduces this further to 128 feature maps at 7×7, followed by a linear layer with a sigmoid output. LeakyReLU with a slope of 0.2 is used in D instead of using ReLU, as it allows small gradients to flow for negative activations, reducing the risk of dead neurons. No BatchNorm is applied to D's first convolutional layer, following standard DCGAN practice, as applying it to the raw input has been found to introduce training instability.

Training images were renormalised to [−1, 1] using mean=0.5 and std=0.5, matching the tanh output range of G. Both G and D were optimised using Adam 2×10^-4 and Beta1 = 0.5, which stabilises GAN training by reducing oscillations caused by the adversarial update dynamic. D maximises log D(x) + log(1 − D(G(z))). In practice, G uses the non-saturating objective, maximising log D(G(z)), which avoids vanishing gradients early in training when D(G(z)) is near zero. A fixed set of 64 noise vectors was held constant throughout training to produce consistent visualisations of G's progress.

The Task 1 BN classifier was used to probe 500 generated images at epoch 10, 20, and 30. Generated images were rescaled from [−1, 1] and then renormalised using the Task 1 classifier mean (0.2860) and std (0.3530), so that the classifier received inputs in the same distribution it was trained on. Two metrics were recorded, the class histogram (argmax of softmax, measuring diversity) and the maximum softmax probability per image (measuring realism). Additionally, after training, 1000 real and 1000 generated images were scored by D to produce the discriminator score distributions in Figure 7.

Results

Figure 5 shows the G and D BCE loss over 30 epochs. D loss begins low (around 0.17 at epoch 1), rises to around 0.31 by epoch 5, then gradually decreases to roughly 0.24 by epoch 30. It remained consistently below the theoretical optimum of ln 2 ≈ 0.693. G loss follows the opposite trend, started at around 2.50, dipping to below 2.00 before epoch 5 as D briefly became less dominant, then rising gradually to above 2.50 by the end ofepoch 30. This divergence of D loss falling while G rises indicates that D grew progressively stronger relative to G over training and was never outpaced by G improvements.

Figure 6 shows a grid of 64 images produced from the fixed noise vectors after 30 epochs of training. The generated images contain blurred image of Fashion-MNIST shapes such as garments, bags, and footwear.

Figure 7 shows the discriminator score distributions for 1000 real and 1000 generated images. Real images had a mean score of 0.628 with std of 0.257 and generated images had a mean score of 0.144 with std of 0.149. The large separation between distributions confirms that D successfully learned to distinguish real from generated images.

Ideally, both distributions would overlap near 0.5 at equilibrium. The clear gap here is consistent with the loss trends in Figure 5.

Figure 8 ranks generated images by their D score. The top row with 0.8-0.89 d scores shows cleaner, more structured images, while the bottom row with ≈ 0.00 scores shows largely incoherent outputs. This demonstrates that D score is a meaningful proxy for perceptual quality within the generated set and being able to separate real from fake.

Figure 9 shows the classifier-based assessment across epochs 10, 20, and 30. The class histograms (figure row A) show T-shirt/top class consistently assigned the highest proportion of generated images, 14%, 16%, and 17% at epochs 10, 20, and 30 respectively. This exceeds the 10% uniform expectation. Some classes such as Trouser and Sneaker remained near or below 10% at all checkpoints. This persistent skew towards T-shirt/top class suggests mild mode collapse, where G overrepresents visually simpler or more common patterns rather than learning the full class diversity. The max softmax probability histograms (figure row B) show means of 0.90, 0.92, and 0.90 across the three checkpoints, with the overall distributions heavily concentrated near 1.0. This indicates the generated images are coherent enough to be classified with high confidence, but also reflects partial mode collapse. G produces images the classifier confidently assigns to a class, even if they lack fine visual detail.

Discussion

The GAN successfully learned to generate recognisable Fashion-MNIST-like images, but with several limitations.Most notably, D became too dominant throughout training. D loss never approached ln 2, and remained consistently below the D optimum value. When D is too capable, the gradient signal G receives becomes near zero, preventing meaningful weight updates. This is known as the vanishing gradient problem in GANs. D's two convolutional layers with BatchNorm enabled more expressive feature extraction compared to G's simpler two-block upsampling, giving D a consistent advantage.

Replacing the minimax loss with the Wasserstein loss would directly address the vanishing gradient problem. The Wasserstein loss quantifies the distance between the real and generated distributions as the minimum effort required to transform one into the other. Compared to the minimax loss, Wasserstein loss remains informative even when D and

G are far apart, providing G with a gradient rather than a flatten loss surface.

The classifier probe results confirm two further limitations.

First, mode collapse towards T-shirt/top class grew over training, from 14% at epoch 10 and 17% at epoch 30. This suggests G converged on a narrow subset of the distribution rather than the full Fashion-MNIST variety. A conditional GAN would address this issue by conditioning G and D on clas label, ensuring G would generate all ten classes, enforcing output distribution diversity.

Second, the uniformly high maximum softmax probabilities of 0.90 may reflect the classifier's own biases toward certain visual patterns rather than truly measuring realism. A generated image may look like a T-shirt/top class to the classifier but is visually unrecognisable to a human observer. In Figure 8, the high scoring images are cleaner but still exhibit a loss of fine texture detail compared to real shapes.

To tackle with the training instability problem, allowing training for more epochs with LR decay would give G oppotunities to improve after D stailises. To tackle output diversity problem, increasing the latent dimension beyond 32 allowing G to have more representational capacity to generate all classes.

Task 3

Methodology

Task 3 replaces the minimax Binary Cross-Entropy loss from Task 2 with the Wasserstein loss (Arjovsky et al., 2017). All other components, the Generator architecture, data normalisation, latent dimension (32), training duration (30 epochs), and classifier probe schedule are unchanged.

In Task 2, D became too dominant, reducing G's gradient signal. The Wasserstein-1 distance $W = E[C(x)] - E[C(G(z))]$ addresses this issue by quantifying the minimum effort required to transform the generated distribution into the real distribution. Unlike BCE, which saturates when $D(G(z)) \rightarrow 0$, W remains proportional to the distribution distance even when the two distributions do not overlap, giving G a meaningful gradient regardless of how strong C becomes.

D becomes a Critic C with the same convolutional architecture as D in task 2, but with the sigmoid output removed so C returns an unbounded real scalar. The training loop structure fron task 2 is unchanged, so there is one C update per batch, one G update per batch. C is trained to maximise W by minimising −(mean(C(real)) − mean(C(fake))). G minimises −mean(C(G(z))). The Lipschitz constraint required for a valid Wasserstein estimate is enforced by clamping all critic weights to [−0.01, 0.01] after each C update. RMSprop with lr = 5×10^{-5} is used for both networks, because Adam default momentum causes oscillations under weight clipping (Arjovsky et al., 2017).

Results

Figure 10 shows the Wasserstein estimate W and the G loss over 30 epochs. W peaks at ~0.07 by epoch 3 then declines monotonically to ~0.0085 by epoch 30, approaching zero. This contrasts with Task 2 where G loss steadily diverged upward from D loss. Both W and G loss converge near zero together, suggesting C provides G with a diminishing gradient signal as training progresses.

Figure 11 shows a grid of 64 WGAN-generated images from the same fixed noise vectors as Figure 6, enabling direct visual comparison. The outputs are somewhat comparable to task 2, showing some clothing silhouettes with similar blurring, and showing some shapes that do not resemble clothing sihouttes.

Figure 12 shows the WGAN critic score distributions for 1000 real and 1000 generated images. Unlike Figure 7's well-separated BCE distributions (real mean 0.628, fake mean 0.144), the WGAN critic scores are near-identical, a real mean of −0.042 with std of 0.009 and a fake mean of −0.049 with std of 0.010. This near-zero separation indicates C cannot meaningfully distinguish real from generated images by the end of training.

Figure 13 shows the classifier-based assessment at epochs 10, 20, and 30. Comparing directly with Figure 9, the WGAN showed stronger mode collapse at epoch 10. T-shirt/top in epoch 10 reached 20.0% versus 14.0% in task 2. By epoch 30 both settle similarly at ~17%, and mean max-probability was slightly lower in the WGAN (0.844–0.877) than task 2 (0.896–0.919). This suggesting generated images are marginally less classifier-confident.

Discussion

The Wasserstein loss was introduced to provide G with a non-saturating gradient proportional to the distribution distance, directly addressing the dominant-D problem identified in task 2. In theory, W should remain informative even when C is confident, giving G a cleaner signal than BCE. However, in practise, the classifier probe in Figure 13 shows that mode collapse worsened compared to Figure 9. The class distribution became more concentrated at epoch 10 rather than more uniform, leading to a more concentrated class distribution compared to Figure 9.

This outcome can be explained by the usage of weight clipping mechanism. In cell 19 C weight is clamped to [−0.01, 0.01] after every update forces C into a very constrained function space. Rather than learning discriminative features of Fashion-MNIST images, C is restricted to near-linear functions. The gradient C provided to G is too weak and too uniform to guide G toward generating a diversed output. This pushes G to mode collapse so it can most easily satisfy under that shallow gradient. The Wasserstein estimator itself is mathematically sound, but weight clipping undermines the critic it depends on.

Gulrajani et al. (2017) identified this limitation and proposed gradient penalty WGAN-GP as a remedy. Instead of hard-clipping weights, WGAN-GP adds a penalty on the gradient norm of C at points interpolated between real and generated samples. This enforces the Lipschitz constraint without restricting what C can learn, allowing the critic to develop expressive features and provide G with a more meaningful gradient, in turns making mode collapse less likely to happen.